Undiscounted problems, where $\alpha = 1$, usually involve a special cost-free termination state. They are generally more complicated than discounted problems, but the preceding results hold under the assumption that there is a bound on the number of stages needed to reach the termination state, regardless of the choices of the minimizer and the maximizer. This condition is fulfilled, for example, in many computer games, like chess, which will be discussed later in this section.

2.12.2 Minimax Approximation in Value Space and Rollout

The approximation ideas for stochastic optimal control that we have discussed in this chapter are also relevant within the minimax context. In particular, for finite horizon problems, approximation in value space with one-step lookahead applies at state x_k a control

$$\tilde{u}_k \in \arg \min_{u_k \in U_k(x_k)} \max_{w_k \in W_k(x_k,u_k)} \Big[g_k(x_k, u_k, w_k) + \tilde{J}_{k+1}\big(f_k(x_k, u_k, w_k)\big) \Big], \tag{2.115}$$

where $\tilde{J}_{k+1}(x_{k+1})$ is an approximation to the optimal cost-to-go $J^*_{k+1}(x_{k+1})$ from state x_{k+1}.

Rollout is obtained when this approximation is the tail cost of some base policy $\pi = \{\mu_0, \ldots, \mu_{N-1}\}$:

$$\tilde{J}_{k+1}(x_{k+1}) = J_{k+1,\pi}(x_{k+1}).$$

Given π, we can compute $J_{k+1,\pi}(x_{k+1})$ by solving a *deterministic maximization* DP problem with the disturbances $w_{k+1}, \ldots, w_{N-1}$ playing the role of "optimization variables/controls." For finite state, control, and disturbance spaces, this is a longest path problem defined on an acyclic graph, since the control variables $u_{k+1}, \ldots, u_{N-1}$ are determined by the base policy. It is then straightforward to implement rollout: at x_k we generate all next states of the form

$$x_{k+1} = f_k(x_k, u_k, w_k)$$

corresponding to all possible values of $u_k \in U_k(x_k)$ and $w_k \in W_k(x_k, u_k)$. We then run the maximization/longest path problem described above to compute $J_{k+1,\pi}(x_{k+1})$ from each of these possible next states x_{k+1}. Finally, we obtain the rollout control $\tilde{u}_k$ by solving the minimax problem in Eq. (2.115). Moreover, it is possible to use truncated rollout to approximate the tail cost of the base policy $J_{k+1,\pi}(x_{k+1})$.†

Note that like all rollout algorithms, the minimax rollout algorithm is well-suited for on-line replanning in problems where data may be changing or may be revealed during the process of control selection.

† For a more detailed discussion of this implementation, see the author's paper [Ber19b] (Section 5.4).

We mentioned earlier that deterministic problems allow a more general form of rollout, whereby we may use a base heuristic that need not be a legitimate policy, i.e., it need not be sequentially consistent. For cost improvement it is sufficient that the heuristic be sequentially improving. A similarly more general view of rollout is not easily constructed for stochastic problems, but is possible for minimax control.

In particular, suppose that at any state x_k there is a heuristic that generates a sequence of feasible controls and disturbances, and corresponding states,

$$\{u_k, w_k, x_{k+1}, u_{k+1}, w_{k+1}, x_{k+2}, \ldots, u_{N-1}, w_{N-1}, x_N\},$$

with corresponding cost

$$H_k(x_k) = g_k(x_k, u_k, w_k) + \cdots + g_{N-1}(x_{N-1}, u_{N-1}, w_{N-1}) + g_N(x_N).$$

Then the rollout algorithm applies at state x_k a control

$$\tilde{u}_k \in \arg \min_{u_k \in U_k(x_k)} \max_{w_k \in W_k(x_k, u_k)} \Big[g_k(x_k, u_k, w_k) + H_{k+1}\big(f_k(x_k, u_k, w_k)\big) \Big].$$

This does not preclude the possibility that the disturbances $w_k, \ldots, w_{N-1}$ are chosen by an antagonistic opponent, but allows more general choices of disturbances, obtained for example, by some form of approximate maximization. For example, when the disturbance involves multiple components, $w_k = (w_k^1, \ldots, w_k^m)$, corresponding to multiple opponent agents, *the heuristic may involve an agent-by-agent maximization strategy*.

The sequential improvement condition, similar to the deterministic case, is that for all x_k and k,

$$\min_{u_k \in U_k(x_k)} \max_{w_k \in W_k(x_k, w_k)} \Big[g_k(x_k, u_k, w_k) + H_{k+1}\big(f_k(x_k, u_k, w_k)\big) \Big] \leq H_k(x_k).$$

It guarantees cost improvement, i.e., that for all x_k and k, the rollout policy

$$\tilde{\pi} = \{\tilde{\mu}_0, \ldots, \tilde{\mu}_{N-1}\}$$

satisfies

$$J_{k,\tilde{\pi}}(x_k) \leq H_k(x_k).$$

Thus, generally speaking, minimax rollout is fairly similar to rollout for deterministic as well as stochastic DP problems. The main difference with deterministic (or stochastic) problems is that to compute the Q-factor of a control u_k, we need to solve a maximization problem, rather than carry out a deterministic (or Monte-Carlo, respectively) simulation with the given base policy.

Example 2.12.1 (Pursuit-Evasion Problems)

Consider a pursuit-evasion problem with state $x_k = (x_k^1, x_k^2)$, where x_k^1 is the location of the minimizer/pursuer and x_k^2 is the location of the maximizer/evader, at stage k, in a (finite node) graph defined in two- or three-dimensional space. There is also a cost-free and absorbing termination state that consists of a subset of pairs (x^1, x^2) that includes all pairs with $x^1 = x^2$, and possibly some other pairs for which x^1 and x^2 are "close enough" for the pursuer to capture the evader.

The pursuer chooses one out of a finite number of actions $u_k \in U_k(x_k)$ at each stage k, when at state x_k, and if the state is x_k and the pursuer selects u_k, the evader may choose from a known set $X_{k+1}(x_k, u_k)$ of next states x_{k+1}, which depends on (x_k, u_k). The objective of the pursuer is to minimize a nonnegative terminal cost $g(x_N^1, x_N^2)$ at the end of N stages (or reach the termination state, which has cost 0 by assumption). A reasonable base policy for the pursuer can be precomputed by DP as follows: given the current (nontermination) state $x_k = (x_k^1, x_k^2)$, make a move along the path that starts from x_k^1 and minimizes the terminal cost after $N-k$ stages, under the assumption that the evader will stay motionless at his current location x_k^2. (In a variation of this policy, the DP computation is done under the assumption that the evader will follow some nominal sequence of moves.)

For the on-line computation of the rollout control, we need the maximal value of the terminal cost that the evader can achieve starting from every $x_{k+1} \in X_{k+1}(x_k, u_k)$, assuming that the pursuer will follow the base policy (which has already been computed). We denote this maximal value by $\tilde{J}_{k+1}(x_{k+1})$. The required values $\tilde{J}_{k+1}(x_{k+1})$ can be computed by an $(N-k)$-stage DP computation involving the optimal choices of the evader, while assuming the pursuer uses the (already computed) base policy. Then the rollout control for the pursuer is obtained from the minimization

$$\tilde{\mu}_k(x_k) \in \arg \min_{u_k \in U_k(x_k)} \max_{x_{k+1} \in X_{k+1}(x_k, u_k)} \tilde{J}_{k+1}(x_{k+1}).$$

Note that the preceding algorithm can be adapted for the imperfect information case where the pursuer knows x_k^2 imperfectly. This is possible by using a form of assumed certainty equivalence: the pursuer's base policy and the evader's maximization can be computed by using an estimate of the current location x_k^2 instead of the unknown true location.

In the preceding pursuit-evasion example, the choice of the base policy was facilitated by the special structure of the problem. Generally, however, finding a suitable base policy that can be conveniently implemented is an important problem-dependent issue.

Variants of Minimax Rollout

Several of the variants of rollout discussed earlier have analogs in the minimax context, e.g., truncation with terminal cost approximation, multistep

and selective step lookahead, and multiagent rollout. In particular, in the ℓ-step lookahead variant, we solve the ℓ-stage problem

$$\min_{u_k,\mu_{k+1},\ldots,\mu_{k+\ell-1}} \max_{\substack{w_k\in W_k(x_k,u_k)\\ w_t\in W_t(x_t,\mu_t(x_t))\\ t=k+1,\ldots,N-1}} \Big\{g_k(x_k,u_k,w_k)+\sum_{t=k+1}^{k+\ell-1} g_t\big(x_t,\mu_t(x_t),w_t\big) + H_{k+\ell}(x_{k+\ell})\Big\},$$

we find an optimal solution $\tilde{u}_k,\tilde{\mu}_{k+1},\ldots,\tilde{\mu}_{k+\ell-1}$, and we apply the first component $\tilde{u}_k$ of that solution. As an example, this type of problem is solved at each move of some chess programs, where the terminal cost function is encoded through a position evaluator. In fact when multistep lookahead is used, special techniques such as *alpha-beta pruning* may be used to accelerate the computations by eliminating unnecessary portions of the lookahead graph. These techniques are well-known in the context of the two-person computer game methodology, and are used widely in games such as chess.

It is interesting to note that, contrary to the case of stochastic optimal control, there is an on-line *constrained form of rollout* for minimax control. Here there are some additional trajectory constraints of the form

$$(x_0,u_0,\ldots,u_{N-1},x_N)\in C,$$

where C is an arbitrary set. The modification needed is similar to the one of Section 2.5: at partial trajectory

$$\tilde{y}_k=(\tilde{x}_0,\tilde{u}_0,\ldots,\tilde{u}_{k-1},\tilde{x}_k),$$

generated by rollout, we use a heuristic with cost function H_{k+1} to compute the Q-factor

$$\tilde{Q}_k(\tilde{x}_k,u_k)=\max_{w_k,\ldots,w_{N-1}}\Big[g_k(\tilde{x}_k,u_k,w_k) + H_{k+1}\big(f_k(\tilde{x}_k,u_k,w_k),w_{k+1},\ldots,w_{N-1}\big)\Big]$$

for each u_k in the set $\tilde{U}_k(\tilde{y}_k)$ that guarantee feasibility [we can check feasibility here by running some algorithm that verifies whether the future disturbances $w_k,\ldots,w_{N-1}$ can be chosen to violate the constraint under the base policy, starting from $(\tilde{y}_k,u_k)$]. Once the set of "feasible controls" $\tilde{U}_k(\tilde{y}_k)$ is computed, we can obtain the rollout control by the Q-factor minimization:

$$\tilde{u}_k\in\arg\min_{u_k\in\tilde{U}_k(\tilde{y}_k)}\tilde{Q}_k(\tilde{x}_k,u_k).$$

We may also use fortified versions of the unconstrained and constrained rollout algorithms, which guarantee a feasible cost-improved rollout policy. This requires the assumption that the base heuristic at the initial state produces a trajectory that is feasible for all possible disturbance sequences.

Truncated and multiagent versions of the minimax rollout algorithm are possible. The following example describes the multiagent case.

Example 2.12.2 (Multiagent Minimax Rollout)

Let us consider a minimax problem where the minimizer's choice involves the collective decision of m agents, $u = (u^1, \ldots, u^m)$, with u^ℓ corresponding to agent ℓ, and constrained to lie within a finite set U^ℓ. Thus u must be chosen from within the set

$$U = U^1 \times \ldots \times U^m,$$

which is finite but grows exponentially in size with m. The maximizer's choice w is constrained to belong to a finite set W. We consider multiagent rollout for the minimizer, and for simplicity, we focus on a two-stage problem. However, there are straightforward extensions to a more general multistage framework.

In particular, we assume that the minimizer knowing an initial state x_0, chooses $u = (u^1, \ldots, u^m)$, with $u^\ell \in U^\ell$, $\ell = 1, \ldots, m$, and a state transition

$$x_1 = f_0(x_0, u)$$

occurs with cost $g_0(x_0, u)$. Then the maximizer, knowing x_1, chooses $w \in W$, and a terminal state

$$x_2 = f_1(x_1, w)$$

is generated with cost

$$g_1(x_1, w) + g_2(x_2).$$

The problem is to select $u \in U$, to minimize

$$g_0(x_0, u) + \max_{w \in W} \big[g(x_1, w) + g_2(x_2)\big].$$

The exact DP algorithm for this problem is given by

$$J_1^*(x_1) = \max_{w \in W} \Big[g_1(x_1, w) + g_2\big(f_1(x_1, w)\big)\Big],$$

$$J_0^*(x_0) = \min_{u \in U} \Big[g_0(x_0, u) + J_1^*\big(f_0(x_0, u)\big)\Big].$$

This DP algorithm is computationally intractable for large m. The reason is that the set of possible minimizer choices u grows exponentially with m, and for each of these choices the value of $J_1^*\big(f_0(x_0, u)\big)$ must be computed.

However, the problem can be solved approximately with multiagent rollout, using a base policy $\mu = (\mu^1, \ldots, \mu^m)$. Then the number of times

$J_1^*\big(f_0(x_0,u)\big)$ needs to be computed is dramatically reduced. This computation is done sequentially, one-agent-at-a-time, as follows:

$$\tilde{u}^1 \in \arg\min_{u^1\in U^1}\Big[g_0\big(x_0,u^1,\mu^2(x_0),\ldots,\mu^m(x_0)\big) + J_1^*\Big(f_0\big(x_0,u^1,\mu^2(x_0),\ldots,\mu^m(x_0)\big)\Big)\Big],$$

$$\tilde{u}^2 \in \arg\min_{u^2\in U^2}\Big[g_0\big(x,\tilde{u}^1,u^2,\mu^3(x_0),\ldots,\mu^m(x_0)\big) + J_1^*\Big(f_0\big(x_0,\tilde{u}^1,u^2,\mu^3(x_0),\ldots,\mu^m(x_0)\big)\Big)\Big],$$

$$\cdots \quad \cdots \quad \cdots \quad \cdots$$

$$\tilde{u}^m \in \arg\min_{u^m\in U^m}\Big[g_0\big(x_0,\tilde{u}^1,\tilde{u}^2,\ldots,\tilde{u}^{m-1},u^m\big) + J_1^*\Big(f_0\big(x_0,\tilde{u}^1,\tilde{u}^2,\ldots,\tilde{u}^{m-1},u^m\big)\Big)\Big].$$

In this algorithm, the number of times for which $J_1^*\big(f_0(x_0,u)\big)$ must be computed grows linearly with m.

When the number of stages is larger than two, a similar algorithm can be used. Essentially, the one-stage maximizer's cost function J_1^* must be replaced by the optimal cost function of a multistage maximization problem, where the minimizer is constrained to use the base policy (see also the paper [Ber19b], Section 5.4).

An interesting question is how do various algorithms work when approximations are used in the min-max and max-min problems? We can certainly improve the minimizer's policy *assuming a fixed policy for the maximizer*; this will be the basis for the scheme of the next section. However, it is unclear how to improve both the minimizer's and the maximizer's policies simultaneously. In practice, in *symmetric games*, like chess, a common policy is trained for both players. In particular, in the AlphaZero and TD-Gammon programs this strategy is computationally expedient and has worked well. However, there is no reliable theory to guide the simultaneous training of policies for both maximizer and minimizer, and it is quite plausible that unusual behavior may arise in exceptional cases.†

Another interesting fact is that even the *exact* policy iteration method (given in 1969 by Pollatschek and Avi-Itzhak [PoA69]) encounters serious convergence difficulties. The reason is that Newton's method applied to solution of the Bellman equation for minimax problems (which is equivalent to policy iteration) exhibits more complex and unreliable behavior than

† Indeed such exceptional cases have been reported for the AlphaGo program in late 2022, when humans defeated an AlphaGo look-alike, KataGo, "by using adversarial techniques that take advantage of KataGo's blind spots" (according to the reports); see Wang et al. [WGB22].

its one-player counterpart. Mathematically, this is because the Bellman operator T for infinite horizon problems, given by

$$(TJ)(x) = \min_{u \in U(x)} \max_{w \in W(x,u)} \Big[g(x,u,w) + \alpha J\big(f(x,u,w)\big) \Big], \qquad \text{for all } x,$$

is neither convex nor concave as a function of J. To see this, note that the function

$$\max_{w \in W(x,u)} \Big[g(x,u,w) + \alpha J\big(f(x,u,w)\big) \Big],$$

viewed as a function of J [for fixed (x,u)], is convex, and when minimized over $u \in U(x)$, it becomes neither convex nor concave. As a result there are special difficulties in connection with convergence of Newton's method and the natural form of policy iteration. The author's paper [Ber21c] and book [Ber22b] (Chapter 5) address these convergence issues with modified versions of the policy iteration method, and give many earlier references on policy iteration and other related methods for minimax problems.

2.12.3 Combined Approximation in Value and Policy Space for Minimax Control

We will now discuss ways to simplify the approximation in value space scheme of the preceding section. For an infinite horizon problem, this scheme applies at state x the control

$$\tilde{u} \in \arg \min_{u \in U(x)} \max_{w \in W(x,u)} \Big[g(x,u,w) + \alpha \tilde{J}\big(f(x,u,w)\big) \Big], \tag{2.116}$$

where $\tilde{J}$ is an approximation to the optimal cost function J^*. The difficulty here is that the maximization over $w \in W(x,u)$ must be performed for each $u \in U(x)$, and complicates the minimization over $u \in U(x)$.

In an alternative scheme that alleviates this difficulty, we may introduce an approximation of the maximizer's optimal policy in addition to the cost function approximation $\tilde{J}$. This is a policy $\nu(x,u)$ that depends on both x and u, and is used to approximate the minimax scheme of Eq. (2.116) with the minimization scheme

$$\tilde{u} \in \arg \min_{u \in U(x)} \Big[g\big(x,u,\nu(x,u)\big) + \alpha \tilde{J}\Big(f\big(x,u,\nu(x,u)\big)\Big) \Big]. \tag{2.117}$$

One possibility to obtain ν is off-line training, using samples from the maximization

$$\max_{w \in W(x,u)} \Big[g(x,u,w) + \alpha \tilde{J}\big(f(x,u,w)\big) \Big],$$

of Eq. (2.116). If the policy $\nu(x,u)$ is optimal for the maximizer, this scheme coincides with the minimax approximation in value space scheme

of the preceding section and inherits the corresponding Newton step-related superlinear cost improvement guarantees.† Otherwise, cost improvement is not guaranteed, but can be expected under favorable conditions where $\nu(x,u)$ is near-optimal.

Once $\nu(x,u)$ has been determined, the simplified scheme (2.117) is just a one-player approximation in value space for a problem defined by the system equation

$$x_{k+1} = f\big(x_k, u_k, \nu(x_k, u_k)\big), \tag{2.118}$$

and the cost per stage

$$g\big(x, u, \nu(x,u)\big).$$

It is important to note that this problem is deterministic, and is suitable for the special search techniques and simplifications that we have discussed in connection with deterministic problems, including the rollout schemes of Sections 2.3-2.6. Moreover the superlinear error bounds associated with Newton's method (cf. Section 1.5.3), apply to this problem.

Extension to Distributionally Robust Control Models

There is an important minimax-type model where the disturbance w_k is a probability distribution and the set $W_k(x_k, u_k)$ is a given set of probability distributions for each pair (x_k, u_k). This problem formulation is known as a *distributionally robust model*. It dates to the early days of statistics (see e.g., the classical book by Blackwell and Girshick [BlG54], and the more recent book by Chen and Paschalidis [ChP20]). Within the RL context of this section, it can be treated as a special case of the schemes that we have discussed. In particular, by allowing the policy $\nu(x,u)$ to take probability distribution values, the approximation in value space scheme (2.117) applies. Note, however, that in this case, the dynamic system (2.118) is stochastic, and as a result the implementation of the corresponding approximation in value space scheme is more complicated.

2.12.4 A Meta Algorithm for Computer Chess Based on Reinforcement Learning

In this section, we will apply the approach of approximating the maximizer's action with a suitably constructed policy to computer chess, based

† A subtle but important point here is that in single-player DP contexts [cf. Eq. (2.117)], the Bellman equation has concave components. This turns out to have a beneficial effect on the convergence properties of Newton's method. By contrast, in two-player games the components of Bellman's equation may be neither concave nor convex, and this complicates significantly the convergence properties of Newton's method; see the author's paper [Ber21b] and abstract DP book [Ber22b] (Chapter 5).